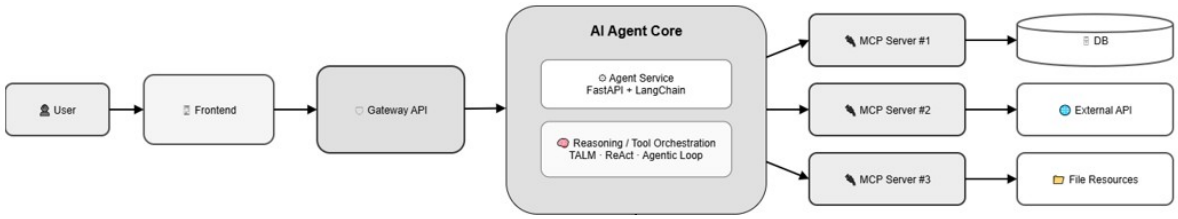
테스트 AI Agent 실행 가이드

1. 각 구성 컴포넌트 소개

- Architecture



- 구성 요소

구분	역할	주요 기술
Frontend	채팅형 UI 를 통해 사용자 입력을 수집하고, AI Agent 로부터 전달된 응답을 사용자에게 시각적으로 표시하는 역할	**React.js,** vue.js
Gateway	사용자 요청에 대한 인증 및 접근 제어를 수행하고, 요청·응답에 대한 로깅과 공통 보안 정책을 적용하는 진입 지점 역할	**Spring Boot**, Spring Security,
MCP Server	AI Agent 의 Tool 호출 요청을 실제 외부 시스템 실행으로 연결하고, 그 결과를 모델 컨텍스트로 변환하여 제공하는 중간 서버 역할	**Spring Boot**, **MyBatis**/JPA
AI Agent Core	대규모 언어 모델(LLM)을 기반으로 사용자 요청을 해석하고, Context 를 유지하며 적절한 Tool 호출 여부를 판단하고 실행을 지시하는 추론 중심 역할, Tool 실행, Context 유지	**Python**, **LangChain**, **FastAPI**
Resource (DB / API / FILE)	MCP Server 를 통해 접근되는 데이터 소스	**PostgreSQL**, REST API, Filesystem

2. 실행 환경 및 설치 가이드

본 프로젝트는 Docker 기반으로 실행되며,
Windows 환경에서는 WSL2 가 필수입니다.
GPU 가 없을 경우 LLM 응답 속도가 현저히 느릴 수 있습니다.

※ **WSL 이란?**
WSL(Windows Subsystem for Linux) 은
Windows 에서 **리눅스를 가볍게 실행하기 위한 기능**

- Docker Desktop 은 WSL2 위에서 실행됨
- 즉, WSL2 가 없으면 Docker 가 정상 동작하지 않음

보통은 docker Desktop 설치 시 함께 설치됨

wsl --status 로 확인 후 설치되어 있지 않으면 **wsl –install** 설치 진행

2.1. 실행 환경(window 기준)

필수

- Windows 10/11 (64bit)
- Docker Desktop (WSL2 backend)
- WSL2 활성화
- Memory 16GB 이상 (Docker 에 12GB 이상 할당)

선택

- NVIDIA GPU (CUDA 지원)

→ 없으면 LLM 응답 속도 매우 느림 (CPU 추론 진행)

- 최신 NVIDIA 드라이버
- WSL GPU 지원 활성화

(GPU 사용여부 테스트 → wsl 에서 **nvidia-smi** 입력 시 GPU 정보 나오면 성공)

2.2. 실행 방법

1. **Docker (Docker Desktop) 설치**
   A. C:\Users\<사용자명>에 있는 .wslconfig 에 memory=12GB 입력 후 저장
   (docker memory 12GB 로 확장)
   B. 최근 Docker Desktop 설치 시 WSL2 자동 설치 및 활성화
   (Use WSL 2 instead of Hyper-V (recommended) 체크박스 체크)
2. **GitLab 에서 해당 프로젝트 파일 다운 및 압축해제**
3. **CMD(명령 프롬프트)에서 프로젝트 경로로 이동한 뒤 다음 명령을 실행**
   (docker-compose up -d –build, 이후 실행 시 docker-compose up -d)
4. **ollama 및 다른 서비스에서 실행 에러가 발생할 경우 (아닐 경우는 성공)**
   A. docker-compose 를 실행했다면 docker-compose down 명령어로 종료
   B. docker-compose up -d ollama 명령어를 통해 ollama 서비스만 실행
   C. docker exec -it ollama ollama pull <사용할 LLM 모델> 해당 명령어로 미리 LLM 설치
   D. docker-compose up -d 으로 전체 서비스 재시작
   (Memory 이슈로 인해 LLM 을 먼저 설치하면 동작 가능)
5. **브라우저에서 http://localhost:3000(frontend) 실행**

2.3. 개발 환경 (window 기준)

로컬 환경에서 테스트 진행 시, 아래와 같은 개발 환경에서 작업 가능합니다.

- frontend

항목	개발도구
개발언어	React.js(Node.js 18+)
패키지 관리자	npm
IDE	vscode

- gateway

항목	개발도구
개발언어	Java 17+ (JDK 17)
프레임워크	Spring Boot 3.5.x
빌드도구	Maven 또는 Gradle (Maven 기준 예시) - 3.9.x
WAS	Spring Boot 내부 tomcat
IDE	intelliJ or Eclipse

- agent

항목	개발도구	비고
개발언어	Python 3.1x	
프레임워크	LangChain	.
WAS / Web	FastAPI / Uvicorn	Uvicorn 은 ASGI 를 구현한 Web 서버 FastAPI 가 실질적 WAS 지만 listen / accept / socket 처리 기능 없음 FastAPI 는 실행 주체가 아니라 실행 “대상”이고, Uvicorn 이 실행 주체다(uvicorn 으로 실행)
IDE	vscode	

- mcp-server

항목	개발도구
개발언어	Java 17+ (JDK 17)
프레임워크	Spring Boot 3.5.x
빌드도구	Maven 또는 Gradle (Maven 기준 예시) - 3.9.x
WAS	Spring Boot 내부 tomcat
IDE	intelliJ or Eclipse

- ollama

wsl 환경에서 다음 명령어를 수행합니다.

1. Ollama 설치 (WSL 내부에서)
   A. curl -fsSL https://ollama.com/install.sh | sh
2. Ollama 실행
   A. ollama serve
3. 모델 다운로드 테스트
   A. ollama pull llama3.1:8b
   B. ollama run llama3.1:8b

- DB / Redis
프로젝트를 다운로드 및 압축 해제 후 CMD 에서 해당 프로젝트 / 경로로 이동한 후
docker-compose up -d postgres redis 명령어를 통해 postgres 와 redis 를 실행시킵니다
(docker-compose.xml 에 있는 postgres 와 redis 만 실행)

3. Docker compose 구성

3.1. frontend

3.1.1. docker-compose.xml

```yaml
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  image: my-frontend:latest
  container_name: frontend
  ports:
    - "3000:80"
  restart: always
```

- --build 옵션 사용 시 Dockerfile 기준으로 gateway 이미지 빌드 수행

3.1.2. Dockerfile

```dockerfile
# build 단계
FROM node:18-alpine AS build
WORKDIR /app

# Docker 용 환경변수 선택
COPY .env.docker .env
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# run 단계
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx","-g","daemon off;"]
```

- node.js 기반으로 프론트엔드 빌드 수행
- Docker 이미지 빌드 시 사용하는 환경설정 파일(.env.docker) 적용
- Nginx 웹 서버를 통해 빌드 결과물 제공

3.2 gateway

3.2.1 docker-compose.xml

```yaml
gateway:
  build:
    context: ./gateway
    dockerfile: Dockerfile
  image: my-gateway:latest
  container_name: gateway
  ports:
    - "8080:8080"
  environment:
      SPRING_PROFILES_ACTIVE: docker
    depends_on:
      - frontend
    restart: always
```

- --build 옵션 사용 시 gateway 서비스의 Dockerfile 기준으로 이미지 빌드 수행
- SPRING_PROFILES_ACTIVE=docker 설정을 통해 application-docker.yml 적용

3.2.2 Dockerfile

```dockerfile
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:17-jre
COPY --from=build /build/target/gateway.jar gateway.jar
ENTRYPOINT ["java","-jar","/gateway.jar"]
```

- Docker 이미지 빌드 시 Maven 기반 프로젝트 빌드 및 패키징 수행
- JRE 17 환경에서 패키징된 Spring Boot JAR 실행

3.3 mcp-server

3.3.1 docker-compose.xml

```yaml
mcp-server:
  build:
    context: ./mcp-server
    dockerfile: Dockerfile
  image: my-mcp-server:latest
  container_name: mcp-server
  ports:
    - "9090:9090"
  environment:
    SPRING_PROFILES_ACTIVE: docker
  depends_on:
    - postgres
  restart: always
```

- --build 옵션 사용 시 mcp-server 서비스의 Dockerfile 기준으로 이미지 빌드 수행
- SPRING_PROFILES_ACTIVE=docker 설정을 통해 application-docker.yml 적용

3.3.2 Dockerfile

```dockerfile
# build 단계
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

# run 단계
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /build/target/mcp-server.jar mcp-server.jar
ENTRYPOINT ["java","-jar","/app/mcp-server.jar"]
```

- Docker 이미지 빌드 시 Maven 기반 프로젝트 빌드 및 패키징 수행
- JRE 17 환경에서 패키징된 Spring Boot JAR 실행

3.4 agent

3.4.1 docker-compose.xml

```yaml
agent:
  build:
    context: ./agent
    dockerfile: Dockerfile
  image: my-agent:latest
  container_name: agent
  ports:
    - "8000:8000"
  env_file:
  - ./agent/.env
  environment:
    REDIS_HOST: redis
    REDIS_PORT: 6379
    REDIS_DB: 0
    REDIS_PASSWORD: wisenut1!
    LLM_MODEL: llama3.1:8b
    MCP_SERVER_URL: http://mcp-server:9090
    OLLAMA_BASE_URL: http://ollama:11434
  depends_on:
    - redis
    - gateway
```

- --build 옵션 사용 시 agent 서비스의 Dockerfile 기준으로 이미지 빌드 수행
- env_file 및 environment 를 통해 환경 변수 설정
- Docker Compose 내부 네트워크에서 서비스 명을 호스트로 사용해 통신
- 사용할 LLM 모델(llama3.1:8b) 환경 변수로 지정

3.4.2 Dockerfile

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn","app.main:app","--host","0.0.0.0","--port","8000"]
```

- Docker 이미지 빌드 시 requirements.txt 기반으로 Python 라이브러리 설치
- ASGI 기반 Web 서버(Uvicorn)로 FastAPI 서비스 실행
- Python 3.11 slim 이미지 기반

3.5 redis

3.5.1 docker-compose.xml

```yaml
redis:
  image: redis:alpine
  container_name: my-redis
  ports:
    - "6379:6379"  # 호스트:컨테이너
  command: redis-server /usr/local/etc/redis/redis.conf
  volumes:
    - ./redis_data:/data # 데이터 보존을 위한 볼륨 마운트
    # [추가] 내 컴퓨터의 redis.conf 를 컨테이너 내부로 연결
    - ./redis.conf:/usr/local/etc/redis/redis.conf
  restart: always
```

- Redis 를 캐시 전용으로 사용하기 위해 데이터 영속화를 비활성화하고, tmpfs 기반으로 Redis 데이터를 관리하여 서비스 재기동 및 컨테이너 종료 시 메모리가 자동 초기화되도록 구성
- 호스트의 redis.conf 파일을 컨테이너 내부 /usr/local/etc/redis/redis.conf 경로에 마운트하여 해당 설정 파일 기준으로 Redis 서버 실행

3.6 PostgreSQL

3.6.1 docker-compose.xml

```yaml
postgres:
  image: postgres:15-alpine
  container_name: my-postgres
  ports:
    - "5432:5432"
  environment:
    POSTGRES_USER: wisenut
    POSTGRES_PASSWORD: wisenut1!
    POSTGRES_DB: data_info
  volumes:
    - ./pg_data:/var/lib/postgresql/data
  restart: always
```

- PostgreSQL 데이터 영속화를 위해 호스트의 ./pg_data 디렉토리를 컨테이너 /var/lib/postgresql/data 에 마운트

3.7 Ollama

3.7.1 docker-compose.xml

```yaml
ollama:
  image: ollama/ollama:latest
  container_name: ollama
  ports:
    - "11434:11434"
  volumes:
    - ollama_data:/root/.ollama
  runtime: nvidia
  environment:
    NVIDIA_VISIBLE_DEVICES: all
  shm_size: "2gb"
  restart: always
```

- Ollama 공식 이미지 사용
- Docker 가 관리하는 네임드 볼륨(ollama_data)을 컨테이너 내부 /root/.ollama 경로에 마운트하여, 다운로드한 LLM 모델 데이터를 저장하고 컨테이너 삭제 및 재생성 시에도 모델 데이터를 유지
- 해당 볼륨은 로컬 프로젝트 디렉토리가 아닌 Docker(WSL2) 내부에 저장되며, Docker 가 실제 저장 위치를 관리함
- NVIDIA GPU 사용 및 모든 GPU 접근 허용 (nvidia-container-toolkit 설치 전제)

※ 네임드 볼륨(Named Volume)

- Docker 가 직접 생성·관리하는 **영속적 데이터 저장 공간**
- 컨테이너 수명과 분리되어 **컨테이너 삭제 후에도 데이터 유지**